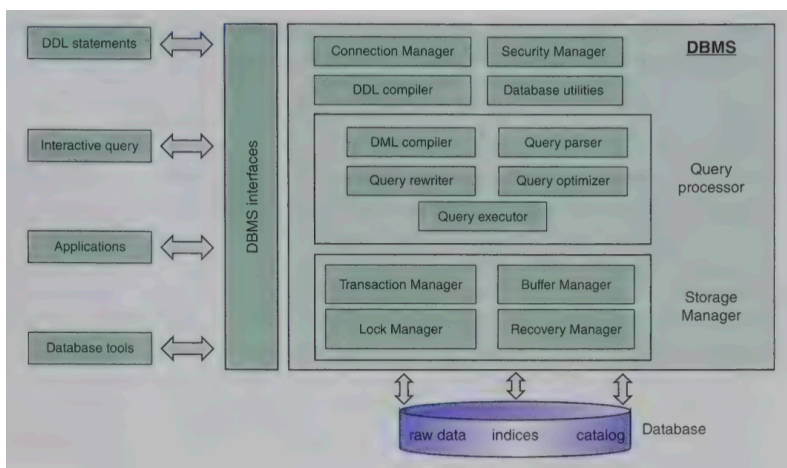


08 Min/Max Summaries & Join Algorithms

▼ Architecture of a Database

General

- A **Database Management System (DBMS)** consists of multiple components that interact to **facilitate database access**, security, processing, and storage.



Core components

- **Connection Manager**
- **Security Manager**
- **DDL Compiler**
- **Query Processor**
- **Storage Manager**
- **Various Database Utilities**
- **DBMS Interfaces**

Core components

1. Connection and Security Manager

- **Connection Manager:** Handles **database connection requests** (local or network-based), verifies user credentials, and returns a connection handle.
- **Security Manager:** Checks user privileges and retrieves access control details from the **catalog**.

2. DDL Compiler

- **Role:** Processes Data Definition Language (DDL) statements for schema creation.
- **Functionality:**
 - Parses DDL syntax.
 - Translates data definitions into an internal format.
 - Registers definitions in the **catalog**.

3. Query Processor

- **Handles** SQL queries for data retrieval, modification, and deletion.
- **Components:**
 - **DML Compiler:** Compiles Data Manipulation Language (DML) statements.
 - **Query Parser:** Checks syntax and converts queries into an intermediate representation.
 - **Query Rewriter:** Optimizes queries by simplifying expressions.
 - **Query Optimizer:** Selects the most efficient execution plan.
 - **Query Executor:** Executes the optimized query.

4. Storage Manager

- **Manages data persistence and retrieval.**
- **Components:**
 - **Transaction Manager:** Ensures ACID properties for transactions.
 - **Buffer Manager:** Handles memory management for efficient access.
 - **Lock Manager:** Maintains concurrency control.
 - **Recovery Manager:** Ensures database integrity after failures.

5. DBMS Utilities

- Includes **backup and recovery tools**, **index management**, and **performance monitoring utilities**.

6. DBMS Interfaces

- Provides different ways for users and applications to interact with the database:
 - **SQL command-line interfaces**
 - **Graphical user interfaces (GUI)**
 - **Application programming interfaces (API) for programmatic access**
 - **Embedded SQL in applications**

▼ Efficient Query Execution

Pruning Large Sets of Files

a			b		
ID	Name		ID	Name	
1	Pinar	File 1	3	Dovile	File 4
2	Veronika		5	Eleni	
3	Dovile		6	Kasper	File 5
4	Martin	File 2	4	Martin	
5	Eleni		1	Pinar	File 6
6	Kasper	File 3	2	Veronika	

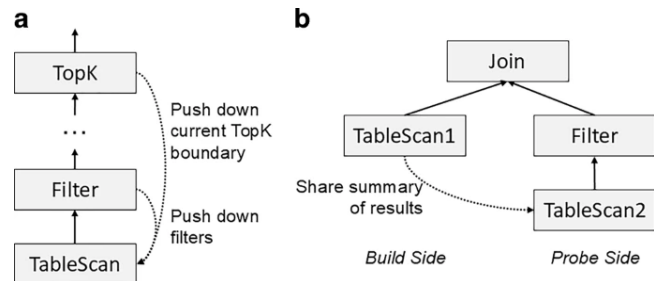
Example table consisting of user data. The **table has two columns, ID and Name**, and **six rows**. The data is sorted by **ID (a)** and by **Name (b)**. The table is partitioned into three files, respectively

The data is split into three files. Each file contains a varying number of rows, **depending** on row size and **partitioning logic** during data modifications.

- To **minimize the number of files** retrieved during query execution, the system employs **pruning techniques using metadata such as min-max indexes for each file**.
 - Creates a **min/max summary** for **each row group**
 - Stores the data in the footer

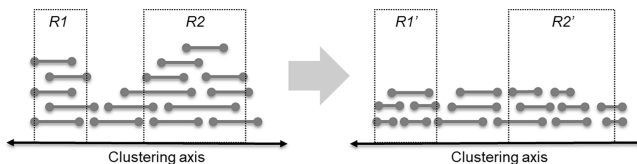
- For instance, if a **query includes a predicate on a specific column**, files whose **min-max ranges** fall outside this predicate can be **excluded**
 - Thereby **reducing the data scanned** and improving performance.

Runtime pruning by pushing down filters and TopK boundaries into the scan operator (a) and sharing a summary of results between scan operators (b)



Optimizing Data Layouts in the Background

- The system **continuously analyzes query patterns** and **reorganizes data storage layouts** accordingly.
- This background optimization ensures that **data is stored in a manner that aligns with common query predicates**, enhancing **data retrieval efficiency** without impacting foreground operations.



Clustering two separate regions, **R1** and **R2**, of a table by **sorting the respective data files along the clustering axis**. The new regions have a **lower clustering depth** than the old regions

Reusing Work:

- To **further boost efficiency**, the system **caches results of previous computations**, materialized indexes, and other reusable artifacts.
- This approach allows the system to **leverage prior work**, **reducing redundant processing** and **accelerating query response times**.

▼ Join Implementation

Table R		Table S	
Employee	Payscale	Payscale	Salary
Cooper	1	1	10000
Gallup	2	2	20000
O'Donnell	1		
Smith	2		

$$R \bowtie S$$

$$r(payscale) = s(payscale)$$

Employee	Payscale	Salary
Cooper	1	10000
Gallup	2	20000
O'Donnell	1	10000
Smith	2	20000

- Joins in SQL are used to **combine rows from multiple tables** based on a related column.
- **Types of joins:**
 - **Inner Join:** Returns only matching rows.
 - **Outer Joins (Left, Right, Full):** Include unmatched rows from one or both tables.
- **Join operations are computationally expensive**, making implementation choices crucial for performance optimization.

Join Implementation Techniques

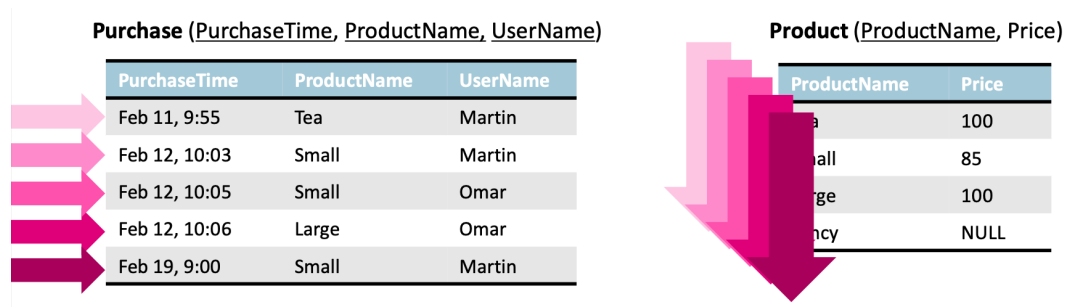
1. Nested-Loop Join

- **Concept:** Iterates over each row in one table and compares it to all rows in another table.
- **Algorithm:**

```
FOR each row s in S DO
  FOR each row r in R DO
    IF r(a) = s(b) THEN join r with s;
```

Best suited for:

- Small inner tables.
- Scenarios where the inner table has an **index on the join column**.



- **Runtime:**
 - $5 * 4$ comparisons $O(N * M)$
 - Quadratic time

2. Sort-Merge Join

- **Concept:** Both tables are first **sorted** on the join attribute before merging.
- **Algorithm:**

1. Sort R on attribute a.
2. Sort S on attribute b.
3. Traverse both tables:
 - If $r(a) < s(b)$, move to next row in R.
 - If $r(a) > s(b)$, move to next row in S.
 - If $r(a) = s(b)$, join and store result

Best suited for:

- Large datasets without indexes.
- When tables are **already sorted on the join attribute**.

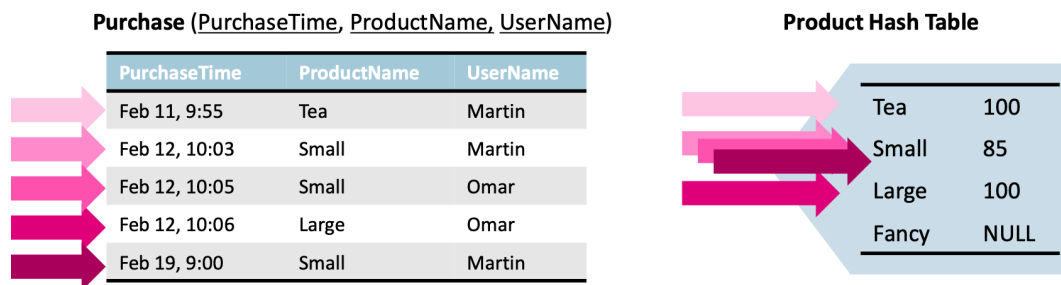
3. Hash Join

- **Concept:** Uses **hashing** to efficiently partition and join tables.
 - Build a hashtable (Datastructure, that allows you to quickly lookup using key)
- **Algorithm:**

Apply a hash function to table R on the join attribute to create partitions.
Apply the same hash function to table S.
Match corresponding partitions and return results.

Best suited for:

- Large datasets.
- Scenarios where join columns are **not sorted and do not have indexes**.



Runtime:

- 4 hashtable inserts and 5 hash look ups $O(N + M)$
- Linear time

SQL Examples of Joins

Basic Inner Join

```
SELECT R.SUPNR, R.SUPNAME, S.PRODNR, S.PURCHASE_PRICE
FROM SUPPLIER AS R
INNER JOIN SUPPLIES AS S
ON R.SUPNR = S.SUPNR;
```

Join Multiple Tables

```
SELECT R.SUPNR, R.SUPNAME, PO.PONR, PO.PODATE, P.PRODNR, P.PRODN
FROM SUPPLIER R
```

```
JOIN PURCHASE_ORDER PO ON R.SUPNR = PO.SUPNR
JOIN PO_LINE POL ON PO.PONR = POL.PONR
JOIN PRODUCT P ON POL.PRODNR = P.PRODNR;
```

Self-Join Example

```
SELECT R1.SUPNAME, R2.SUPNAME, R1.SUPCITY
FROM SUPPLIER R1, SUPPLIER R2
WHERE R1.SUPCITY = R2.SUPCITY
AND R1.SUPNR < R2.SUPNR;
```

Comparison of Join Methods

Join Type	Best Use Case	Pros	Cons
Nested-Loop	Small inner table or indexed	Simple to implement	Very slow for large tables
Sort-Merge	No index, sorted input	Linear scan for matching tuples	Sorting overhead
Hash Join	Large tables, no sorting Best when both sides are large and unsorted	Fast for large datasets	Memory-intensive (hashing process)

Exercises